Phase 1: BPE 分词器算法原理

1. Unicode 基础

1.1 字符、码点与编码的三层关系

```
字符 (Character)  →  码点 (Code Point)  →  字节序列 (Byte Sequence)
      "牛"        →     U+725B (29275)   →  \xe7\x89\x9b (3 bytes in UTF-8)
```

- **字符**：人类可读的符号
- **码点**：Unicode 标准中给每个字符分配的整数（约 15 万个）
- **编码**：将码点转换为字节序列的规则（UTF-8 / UTF-16 / UTF-32）

1.2 UTF-8 编码规则

UTF-8 是变长编码，1~4 个字节表示一个码点：

码点范围	字节数	字节模式
U+0000 ~ U+007F	1	`0xxxxxxx`
U+0080 ~ U+07FF	2	`110xxxxx 10xxxxxx`
U+0800 ~ U+FFFF	3	`1110xxxx 10xxxxxx 10xxxxxx`
U+10000 ~ U+10FFFF	4	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

关键特性：

- ASCII 字符（英文字母、数字、基础标点）只占 1 字节，与 ASCII 完全兼容
- 中文、日文、韩文通常占 3 字节
- 续字节（continuation byte）总是 `10xxxxxx`（0x80~0xBF），首字节绝不在这个范围
- 这意味着：**不能把多字节字符的单个字节独立解码**

1.3 Python 中的 bytes 操作

```
# str → bytes（编码）
s = "hello 牛"
b = s.encode("utf-8")          # b'hello \xe7\x89\x9b'
print(list(b))                  # [104, 101, 108, 108, 111, 32, 231, 137, 155]

# bytes → str（解码）
b.decode("utf-8")               # "hello 牛"

# 单字节操作
bytes([104])                    # b'h'
bytes([231, 137, 155])          # b'\xe7\x89\x9b' → "牛" 的 UTF-8 编码

# bytes 拼接
b'hello' + b' ' + b'world'     # b'hello world'

# bytes 没有单独的 byte 类型
b'hello'[0]                     # 104 (int, 不是 bytes)
bytes([b'hello'[0]])            # b'h' (要想得到单字节 bytes 需要包装)

# 安全解码（遇到非法字节用 U+FFFD 替代）
b'\xff\xfe'.decode("utf-8", errors="replace")  # '\uFFFD\uFFFD'
```

1.4 为什么选 UTF-8 而不是 UTF-16/UTF-32

- **UTF-32**：每个码点固定 4 字节。英文文本会浪费 3/4 空间，且大量字节值为 0x00（null byte），增加无用 token
- **UTF-16**：每个码点 2 或 4 字节。同样会有大量 0x00 字节（对 ASCII 字符）
- **UTF-8**：
  - 英文文本（互联网主流）紧凑高效
  - 无冗余 null byte
  - 初始词表固定 256，不会浪费

思考题答案方向：用不同编码编码 "hello" 看输出——UTF-32 产生大量 `\x00`，UTF-16 每个 ASCII 字符前有 `\x00`，UTF-8 最紧凑。

2. BPE 训练算法

2.1 算法全貌（伪代码级）

```
输入: 语料文本, 目标词表大小 V, 特殊 token 列表
输出: 词表 vocab, 合并列表 merges

1. vocab = {i: bytes([i]) for i in range(256)}
   为每个 special_token 分配 ID 加入 vocab

2. 预分词:
   用 GPT-2 正则对语料切分成 pre-tokens
   统计每个 pre-token 的出现频次
   将每个 pre-token 转为 UTF-8 字节元组: "the" → (b't', b'h', b'e')
   结果: word_freqs = {(b't',b'h',b'e'): 500, (b' ',b'c',b'a',b't'): 200, ...}

3. 初始化 pair 频率:
   遍历 word_freqs 中每个 (token_tuple, freq):
     对 token_tuple 中每对相邻 token (A, B):
       pair_counts[(A, B)] += freq

4. 循环直到 len(vocab) >= V:
   a. 找到 pair_counts 中频率最高的 pair (A, B)
      若并列 → 选字典序最大的
   b. new_token = A + B  (bytes 拼接)
   c. 把 new_token 加入 vocab
   d. 记录 merge: merges.append((A, B))
   e. 更新 word_freqs 和 pair_counts (关键的增量更新步骤)

返回 vocab, merges
```

2.2 预分词（Pre-tokenization）详解

GPT-2 使用的正则模式：

```
PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

这个模式的各部分含义：

模式	匹配内容	示例
`'(?:[sdmt]|ll|ve|re)`	英文缩写后缀	`'ll`，`'ve`，`'re`，`'s`，`'d`，`'m`，`'t`
`| ?\p{L}+`	可选前导空格 + 连续字母	`some`， `text`， `that`
`| ?\p{N}+`	可选前导空格 + 连续数字	`123`， `456`
`| ?[^\s\p{L}\p{N}]+`	可选前导空格 + 非字母非数字非空白	`-`， `!@#`
`|\s+(?!\S)`	尾部空白（后面没有非空白字符）	文末的空格/换行
`|\s+`	其他空白	单词间的空格

关键效果：

- 空格通常被"粘"在后面的单词前面： `"the cat"` → `["the", " cat"]`
- 标点与字母分开： `"don't"` → `["don", "'t"]`
- 数字和字母分开
- 这样合并时不会跨越自然的词边界

使用 `finditer` 而非 `findall` 的原因：

```
# findall 返回完整列表——大语料会占很多内存
all_tokens = re.findall(PAT, text)

# finditer 返回迭代器——可以边迭代边统计，不存储完整列表
for match in re.finditer(PAT, text):
    token = match.group()
    counts[token] += 1
```

2.3 处理 Special Tokens

在预分词之前，必须先把 special token（如 `<|endoftext|>` ）从文本中分离出来：

```
原文: "[Doc 1]<|endoftext|>[Doc 2]"
      ↓ 按 special token 分割
片段: ["[Doc 1]", "[Doc 2]"]
      ↓ 分别对每个片段做预分词
结果: 对 "[Doc 1]" 和 "[Doc 2]" 各自跑 re.finditer
```

为什么：

1. 不希望 special token 的字节参与 merge（它应始终作为整体）
2. 不希望跨文档边界产生 merge

实现思路：用 `re.split + re.escape` 来按 special token 切分文本。

2.4 增量更新 pair 计数（性能关键）

朴素做法：每次 merge 后重新扫描所有 word_freqs 统计 pair 计数 → O(N) per merge

高效做法：只更新受影响的 pair

当合并 (A, B) → AB 时，对每个包含 (A, B) 的 word：

```
假设某个 word 中有片段: ... X A B Y ...

合并后变为: ... X AB Y ...

需要更新的 pair:
  - 减少: (X, A), (A, B), (B, Y)    ← 这些 pair 的一些出现消失了
  - 增加: (X, AB), (AB, Y)           ← 产生了新的相邻 pair

(A, B) 本身的计数直接清零/移除
```

数据结构建议：

- `word_freqs: dict[tuple[bytes, ...], int]` — 每个 word 的 token 元组 → 频次
- `pair_counts: dict[tuple[bytes, bytes], int]` — pair → 总频次
- 可能还需要一个反向索引： `pair → 包含该 pair 的 word 列表` ，方便定位需要更新的 word

选择最大 pair 的效率：

- 朴素：每次 `max(pair_counts, key=pair_counts.get)` → O(词表大小)
- 可选优化：用堆（heapq）或有序容器，但要注意 lazy deletion
- 对于本作业，朴素 max 通常够用（瓶颈在预分词）

2.5 并行化预分词

```
文件 ─── find_chunk_boundaries() ──→ [0, pos1, pos2, pos3, file_size]
                                          │     │     │     │
                                          ▼     ▼     ▼     ▼
                                       chunk1 chunk2 chunk3 chunk4
                                          │     │     │     │
                                          ▼     ▼     ▼     ▼
                                 (每个进程) 对 chunk 做预分词，返回 Counter
                                          │     │     │     │
                                          └─── 合并 Counter ──→ 总 word_freqs
```

`find_chunk_boundaries` 的原理：

- 均匀分割文件大小
- 每个边界向后搜索到最近的 `<|endoftext|>` 位置
- 这样保证每个 chunk 都完整包含若干文档，不会把一个文档切成两半

3. BPE 编码（Encoding）

3.1 编码算法

给定训练好的 merges 列表和词表，对新文本编码：

```
输入: 文本字符串
输出: token ID 列表

1. 处理 special tokens:
   按 special tokens 分割文本
   对于每个 special token → 直接映射为对应 ID
   对于每个非 special 文本段 → 走下面的流程

2. 预分词: 用同样的 GPT-2 正则切分文本段

3. 对每个 pre-token:
   a. 转为 UTF-8 字节列表: [b't', b'h', b'e']
   b. 按 merges 列表的顺序，依次尝试应用每个 merge:
      对于 merge (A, B):
        扫描当前 token 列表，找到所有相邻的 (A, B)
        将它们合并为 A+B
   c. 合并完成后，查词表得到每个 token 的 ID

4. 拼接所有 pre-token 的 ID 列表 → 最终输出
```

3.2 编码示例演练

```
词表: {0:b' ', 1:b'a', ..., 5:b't', 6:b'th', 7:b' c', 8:b' a', 9:b'the', 10:b' at'}
merges: [(b't',b'h'), (b' ',b'c'), (b' ',b'a'), (b'th',b'e'), (b' a',b't')]

输入: "the cat ate"
预分词: ["the", " cat", " ate"]

处理 "the":
  初始: [b't', b'h', b'e']
  merge (b't',b'h'):  [b'th', b'e']
  merge (b' ',b'c'):  无匹配
  merge (b' ',b'a'):  无匹配
  merge (b'th',b'e'): [b'the']
  → ID: [9]

处理 " cat":
  初始: [b' ', b'c', b'a', b't']
  merge (b't',b'h'):  无匹配
  merge (b' ',b'c'):  [b' c', b'a', b't']
  merge (b' ',b'a'):  无匹配（' '已被合并）
  merge (b'th',b'e'): 无匹配
  merge (b' a',b't'): 无匹配
  → ID: [7, 1, 5]

处理 " ate":
  初始: [b' ', b'a', b't', b'e']
  merge (b't',b'h'):  无匹配
  merge (b' ',b'c'):  无匹配
  merge (b' ',b'a'):  [b' a', b't', b'e']
  merge (b'th',b'e'): 无匹配
  merge (b' a',b't'): [b' at', b'e']
  → ID: [10, 3]

最终: [9, 7, 1, 5, 10, 3]
```

3.3 编码效率考虑

对每个 pre-token 应用所有 merges 的朴素复杂度是 O(merges数 × pre-token长度)。

可以优化：

- 构建 merge 优先级字典 `{(A,B): priority}` 其中 priority 是 merge 的创建顺序
- 对 pre-token 内的所有相邻 pair，找优先级最高（创建最早）的 merge 先应用
- 但对于本作业，按顺序遍历 merges 列表通常足够

3.4 `encode_iterable` 的设计

这个方法接收一个字符串迭代器，返回 token ID 迭代器。核心思想是流式处理：

```
def encode_iterable(self, iterable):
    for text in iterable:
        for token_id in self.encode(text):
            yield token_id
```

用 `yield` 实现 generator，避免一次性把所有文本都 encode 进内存。

4. BPE 解码（Decoding）

4.1 解码算法

```
输入: token ID 列表
输出: 文本字符串

1. 对每个 ID 查词表得到 bytes
2. 按顺序拼接所有 bytes
3. 用 UTF-8 解码: result_bytes.decode("utf-8", errors="replace")
```

解码非常简单，因为词表直接存储了每个 token 对应的字节串。

4.2 为什么需要 `errors='replace'`

某些 token ID 序列在拼接后可能不构成合法的 UTF-8：

- 模型可能生成的 token 恰好对应多字节字符的一部分
- 例如 "牛" 的 UTF-8 是 3 个字节 `[0xe7, 0x89, 0x9b]`，如果模型只生成了前两个字节对应的 token，那就无法解码

此时 `errors='replace'` 会用 `U+FFFD` (` `) 替代非法字节。

5. BPE 训练的完整数据流

```
原始文本文件
    │
    ▼
find_chunk_boundaries() 按 <|endoftext|> 切分 chunk
    │
    ▼ (multiprocessing)
每个 chunk:
    │── 按 special token 分割
    │── 对每段用 re.finditer(PAT, ...) 预分词
    │── 将每个 pre-token 编码为 UTF-8 bytes tuple
    │── 统计频次 → Counter
    │
    ▼ (合并所有 Counter)
word_freqs: dict[tuple[bytes,...], int]
    │
    ▼
初始化 pair_counts
    │
    ▼
循环 merge:
    ├── 找最频繁 pair
    ├── 创建新 token
    ├── 更新 word_freqs 中所有包含该 pair 的 entry
    ├── 增量更新 pair_counts
    └── 记录到 merges 列表
    │
    ▼
输出: vocab (int→bytes), merges (list of (bytes,bytes))
```

6. 易错点与注意事项

1. **bytes 比较与字典序**：Python 的 `bytes` 对象支持 `<` `>` 比较，按字典序（逐字节比较）。`(b'BA', b'A') > (b'B', b'ZZ')` 因为第一个元素 `b'BA' > b'B'`
2. **merge 时的多次出现**：一个 word 中 pair (A, B) 可能出现多次，如 `(b'a', b'b', b'a', b'b')` 中 `(b'a', b'b')` 出现 2 次。合并时要全部替换（从左到右不重叠）

3. **pre-token 转 bytes tuple**： `"the".encode("utf-8")` 得到 `b'the'` ，需要拆成 `(bytes([116]), bytes([104]), bytes([101]))` 即 `(b't', b'h', b'e')`
4. **词表 ID 分配顺序**：special tokens 和初始 256 字节的 ID 分配需要明确。一种常见顺序：special tokens 先分配，然后 0~255 字节，然后 merge 产生的新 token
5. **空 pre-token**：正则匹配不应产生空字符串，但做好防御性检查
6. **大文件处理**：编码大文件时需要流式处理，按 special token 切块，确保 token 不跨 chunk 边界